COMPLETE VISUAL TUTORIAL · BEGINNER → PRO

# Random Forest
## from scratch

How the algorithm works, why it works, the maths behind it, and how to use it like a professional – with diagrams, worked examples and runnable code.

| Part | What you will learn |
|---|---|
| 1 · Basics | What a Random Forest is, why it is named so, the prerequisites (trees, ensembles, bagging) |
| 2 · How it works | Bootstrapping (row / column / combined sampling), aggregation, a hands-on build from scratch |
| 3 · Why it works | Bias–variance tradeoff, variance-reduction maths, visual proofs on classification and regression |
| 4 · Bagging vs RF | The two differences, tree-level vs node-level feature sampling, interview answer |
| 5 · OOB evaluation | Out-of-bag samples, the 36.8 % result, free validation |
| 6 · Feature importance | Why, how to read it, full step-by-step maths for trees and forests |
| 7 · Pro level | Hyper-parameters, tuning, pitfalls, permutation importance, cheat sheet, interview Q&A; |

# Contents


**1. What is a Random Forest? . . . . . 4**

1.1 Why everybody likes it . . . . . 4

1.2 An everyday analogy – the wisdom of the crowd . . . . . 4

1.3 What you should know first (prerequisites) . . . . . 5

**2. Prerequisite refresher . . . . . 5**

2.1 A decision tree in one minute . . . . . 5

2.2 The weakness of a single tree . . . . . 6

2.3 Ensemble learning . . . . . 6

**3. Why is it called 'Random Forest'? . . . . . 7**

3.1 Random Forest = Bagging + Decision Trees (a specialisation) . . . . . 7

**4. How Random Forest works – the big picture . . . . . 8**

**5. Step 1 – Bootstrapping (creating the subsets) . . . . . 8**

5.1 Row sampling . . . . . 8

5.2 Column (feature) sampling . . . . . 9

5.3 Combined sampling . . . . . 9

**6. Step 2 – Aggregation (making the prediction) . . . . . 10**

6.1 Classification – majority vote ('democracy') . . . . . 10

6.2 Regression – average . . . . . 10

6.3 The complete algorithm . . . . . 10

**7. Hands-on: build the idea from scratch . . . . . 11**

7.1 Dataset and the three sampling functions . . . . . 11

7.2 Experiment A – row sampling . . . . . 11

7.3 Experiment B – column sampling . . . . . 12

7.4 Experiment C – combined sampling . . . . . 12

**8. Why does it work? Bias and Variance . . . . . 13**

8.1 Algorithms grouped by their bias/variance behaviour . . . . . 13

**9. How Random Forest reduces variance . . . . . 14**

9.1 The intuition – spreading the noise . . . . . 14

9.2 The mathematics (pro level) . . . . . 14

**10. Seeing it: decision boundaries and regression . . . . . 15**

10.1 Classification on concentric circles . . . . . 15

10.2 Regression on noisy data . . . . . 16

**11. Bagging vs Random Forest . . . . . 17**

11.1 Difference 1 – the base algorithm . . . 17
11.2 Is 'Bagging with Decision Trees' then a Random Forest? NO. . . . 17
11.3 Example with 5 columns and 2 columns per tree . . . 17
**12. OOB (Out-of-Bag) evaluation . . . 19**
12.1 The idea . . . 19
12.2 Why ≈ 37 %? (the maths) . . . 19
12.3 Using it in scikit-learn . . . 19
**13. Feature importance . . . 21**
13.1 What is it – and why do we need it? . . . 21
13.2 Visual example – MNIST digits . . . 21
**14. The maths of feature importance – step by step . . . 22**
14.1 The formulas . . . 22
14.2 Worked example 1 – five rows, two columns . . . 22
14.3 Worked example 2 – three split nodes . . . 23
14.4 From one tree to a forest – just average . . . 24
14.5 A warning – impurity importance can mislead . . . 24
**15. Pro level – hyper-parameters . . . 25**
**16. Tuning and a complete pipeline . . . 26**
**17. Strengths, limits and related models . . . 27**
17.1 Related models . . . 27
**18. Cheat sheet, interview Q&A; and glossary . . . 28**
18.1 One-page summary . . . 28
18.2 Interview questions . . . 28
18.3 Glossary . . . 29

PART 1 · BASICS

# 1. What is a Random Forest?

Random Forest is one of the most popular and dependable machine-learning algorithms. If you work on machine-learning projects you will meet it again and again. It sits in the same top tier as **Gradient Boosting** and **XGBoost**: algorithms whose performance stays strong on almost every kind of problem.

**ONE-LINE DEFINITION**

A Random Forest is a **collection (a forest) of many decision trees**. Each tree is trained on a slightly different random piece of the data, and the final answer is the **vote** (classification) or **average** (regression) of all the trees.

## 1.1 Why everybody likes it

- **Great results almost everywhere** – it performs strongly on nearly every type of machine-learning problem.
- **Works for both tasks** – classification (is this email spam?) *and* regression (what will the house price be?).
- **Needs very little tuning** – even with default settings it often performs very well. In many projects, when you compare it with other algorithms using default parameters, Random Forest is frequently the best or close to the best. That makes it perfect for beginners.
- **Built-in extras** – out-of-bag evaluation (Chapter 12) and feature importance (Chapters 13–14) come almost for free.

**CLEARED UP FROM THE VIDEO**

The video says the creator of Random Forest is unknown. In fact the method was published by **Leo Breiman in 2001**, building on earlier work by Tin Kam Ho (random subspace method) and Amit & Geman, and the modern algorithm is closely tied to Breiman and Adele Cutler. The paper's remark that the algorithm needs little tuning is real – that is why it is called a great 'first model'.

## 1.2 An everyday analogy – the wisdom of the crowd

Imagine you must decide whether to buy a second-hand car. You could ask **one** expert – but he may have a personal bias or a bad day. Or you can ask **100 mechanics**, each of whom has inspected a *different* set of cars in their career, and go with the majority opinion. Individual mechanics make mistakes, but their mistakes are different, so they cancel out when you combine the opinions. That is exactly the principle behind a Random Forest: **many imperfect but different experts beat one expert**.

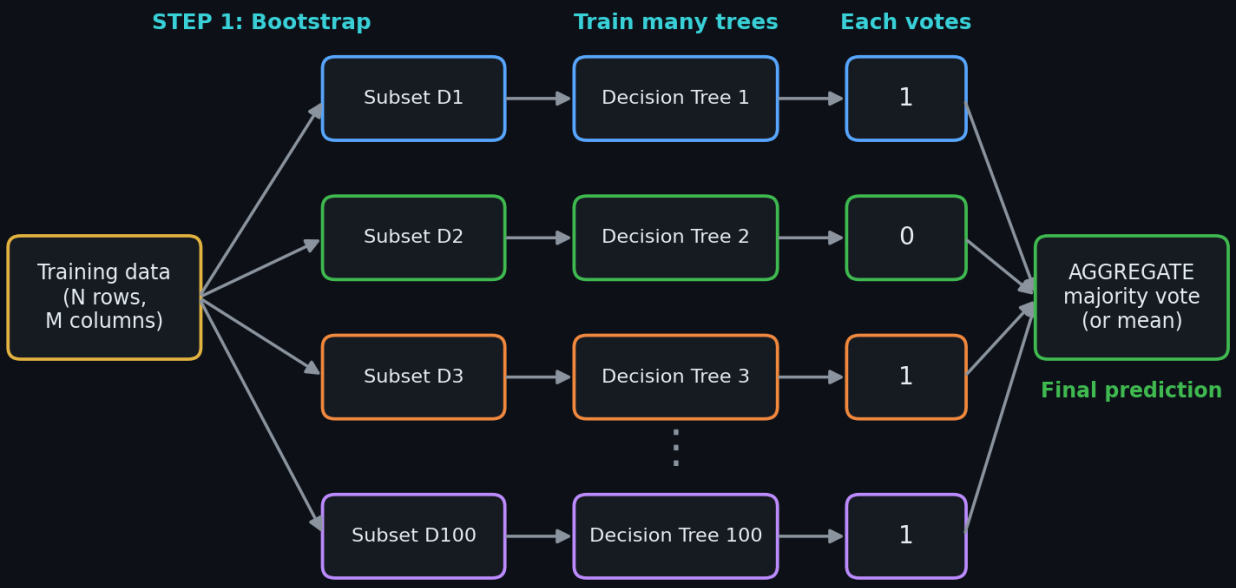



*Figure 1 – The whole algorithm in one picture: sample the data, train one tree per sample, let every tree vote, aggregate.*

## 1.3 What you should know first (prerequisites)

- **Decision Trees** – Random Forest is basically a collection of trees. Without trees you cannot understand the forest (refresher in Chapter 2).
- **Ensemble learning** – combining many models into one stronger model.
- **Bagging** (Bootstrap Aggregation) – Random Forest is a bagging technique. The tutorial explains bagging from zero, so you can follow even if you haven't studied it.

**PART 1 · BASICS**

# 2. Prerequisite refresher

## 2.1 A decision tree in one minute

A decision tree asks a series of yes/no questions about the features and ends in a prediction. Every question is a **node**; the final answers are **leaves**. At each node the tree chooses the question that makes the groups as **pure** as possible (mostly one class).

### Measuring impurity: Gini and Entropy

$$Gini(t) = 1 - \Sigma_k \, p_k^{\,2}$$
$$Entropy(t) = -\Sigma_k \, p_k \cdot \log_2(p_k)$$

Here $p_k$ is the fraction of samples of class $k$ inside the node. A perfectly pure node (all one class) has Gini = 0. A 50/50 two-class node has the highest Gini = 0.5.

**EXAMPLE**

A node holds 5 samples: 2 of class A and 3 of class B.

$p_A = 2/5 = 0.4$, $p_B = 3/5 = 0.6$.

Gini $= 1 - (0.4^2 + 0.6^2) = 1 - (0.16 + 0.36) =$ **0.48**. (We will meet this exact number again in the feature-importance maths.)

The tree tries every possible split and keeps the one with the biggest **impurity decrease** (information gain):

$$Gain = G(parent) - [\ (n_L/n)\cdot G(left) + (n_R/n)\cdot G(right)\ ]$$

## 2.2 The weakness of a single tree

If you let a tree grow without limits (no *max_depth*), it keeps splitting until every training row is classified correctly. It essentially **memorises** the training data, including its noise. On new data it performs poorly. This is called **overfitting** or **high variance**. A Random Forest is the cure for exactly this problem.

## 2.3 Ensemble learning

An **ensemble** combines several models (called *base models* or *base estimators*). Two big families exist:

- **Bagging** – train models in *parallel* on different random samples, then combine. (Random Forest lives here.)
- **Boosting** – train models one after another, each fixing the mistakes of the previous (AdaBoost, Gradient Boosting, XGBoost).

**INFO**

Ensembles work best when the base models are **individually decent** (better than random guessing, i.e. accuracy a bit above 50 %) and **different from each other**. The more different they are, the better the combined result. Keep this sentence in mind – it explains most of Random Forest's design.

PART 1 · BASICS

# 3. Why is it called 'Random Forest'?

The name has two words, and each has a clear reason:

| Word | Reason |
|---|---|
| **Forest** | You train **multiple decision trees** together. A group of trees is a forest. |
| **Random** | Randomness comes from **sampling**: each tree gets a randomly selected subset of rows and/or columns (the 'Bootstrap' in Bootstrap Aggregation). Later we will see even more randomness: random columns at every split. |

## 3.1 Random Forest = Bagging + Decision Trees (a specialisation)

In plain **bagging** you may choose *any* base algorithm – KNN, SVM, logistic regression, anything. When the base algorithm is always a **Decision Tree** (and a few extra tweaks are applied, see Chapter 11), the technique has a special name: **Random Forest**.

| Technique | Base model | Name |
|---|---|---|
| Bagging + KNN | KNN | Bagging classifier |
| Bagging + SVM | SVM | Bagging classifier |
| Bagging + Decision Trees (+ node-level feature sampling) | Decision Tree | **Random Forest** |

PART 2 · HOW IT WORKS

# 4. How Random Forest works – the big picture

Training and prediction happen in **two steps**. Assume a dataset with 1,000 rows and several columns, and a forest of 100 trees.

| Step | Name | What happens |
|---|---|---|
| 1 | **Bootstrapping** (training) | Create 100 different random subsets D1…D100 of the data. Give $D_i$ to Decision Tree i and train it. |
| 2 | **Aggregation** (prediction) | Send a new query point to all 100 trees. Combine their answers: majority vote (classification) or mean (regression). |

**KEY IDEA**

Because every tree sees different data, every tree looks different and makes different mistakes. Averaging many different, individually-good trees cancels the mistakes.

PART 2 · HOW IT WORKS

# 5. Step 1 – Bootstrapping (creating the subsets)

Each base model receives a **subset** of the data instead of everything. There are three ways to build that subset.

## 5.1 Row sampling

Suppose you have 1,000 rows and decide each tree gets 500 rows. Which 500? **Randomly chosen** – perhaps rows 1, 18, 32, 43, 103 … This random subset is called **D1** and goes to Decision Tree 1. D2 is another random subset, and so on.

### With replacement vs without replacement

**With replacement**: after a row is picked it is put back, so it can be picked again. The subset may contain **duplicates**, and some rows may never be picked. **Without replacement**: a picked row is not put back, so there are **no duplicates**.




*Figure 2 – Row sampling. With replacement, 'row 1' and 'row 5' appear twice (red) while rows such as 6, 7, 9 and 10 are never chosen.*

**IN PRACTICE**

In the classic Random Forest (and in scikit-learn with *bootstrap=True*) every tree receives a sample **with replacement** that is by default **the same size as the original data** (N rows drawn from N). The 500-of-1000 example in the video simply shows the idea of a smaller subset – you can control it with the *max_samples* parameter.

## 5.2 Column (feature) sampling

The same idea applied to columns. With 10 columns you might give every tree 5 random columns. This is also called **feature sampling** and it too can be with or without replacement. (Row sampling works on rows, feature sampling works on columns.)

## 5.3 Combined sampling

Do both: take a random subset of rows *and* a random subset of columns. This gives the highest diversity between trees.

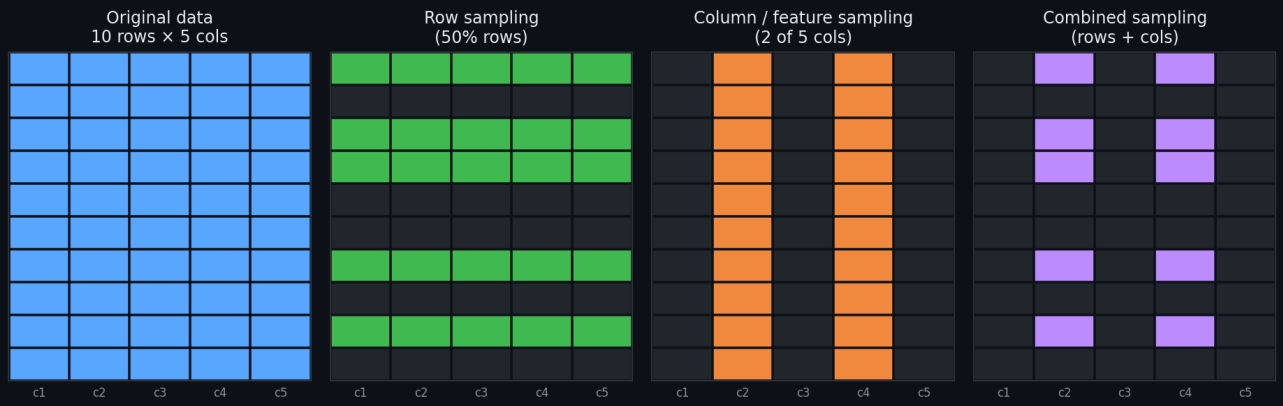



*Figure 3 – The three ways of creating a subset (coloured cells = data the tree is allowed to see).*

Since every tree is fed different data, the trees come out different – one may split first on age, another on income, another on credit score. That diversity is the whole point.

PART 2 · HOW IT WORKS

# 6. Step 2 – Aggregation (making the prediction)

After training, a new query point is passed through **all** trees. Each tree gives its own prediction and we combine them.

## 6.1 Classification – majority vote ('democracy')

Suppose 43 of 100 trees say class 0 and 57 say class 1. Class 1 has more votes, so the forest predicts class 1.

$$\hat{y} = mode\ \{\ h_1(x),\ h_2(x),\ \ldots\ ,\ h_B(x)\ \}$$

## 6.2 Regression – average

Each tree returns a number. The forest returns their mean.

$$\hat{y} = (1\ /\ B)\ \cdot\ \Sigma_{b=1..B}\ h_b(x)$$




*Figure 4 – Aggregation: voting for classification, averaging for regression (here the five trees predict 21.5, 24, 22, 26.5, 23 → mean 23.4).*

*(In scikit-learn the classifier actually averages the trees' predicted **probabilities** and picks the highest – 'soft voting' – which behaves almost the same as a hard majority vote.)*

## 6.3 The complete algorithm

```
TRAINING  (B = number of trees)
for b = 1 ... B:
    1. Draw a bootstrap sample D_b from the training data (rows with replacement)
    2. Grow a decision tree T_b on D_b. At EVERY node:
          - pick m features at random (m = sqrt(M) for classification is the usual default)
          - choose the best split among only those m features
          - split until the tree is fully grown (or a stopping rule is hit)

PREDICTION for a new point x
    classification :  y_hat = majority vote of T_1(x) ... T_B(x)
    regression     :  y_hat = mean of      T_1(x) ... T_B(x)
```

**EXAMPLE**

**Loan approval, three trees, one new applicant.** Applicant: income 60k, credit score 720, age 23.

Tree 1 (rule: credit ≥ 700 → approve) says **approve**. Tree 2 (rule: income ≥ 50k → approve) says **approve**. Tree 3 (rule: age < 25 → reject) says **reject**.

Votes: approve 2, reject 1 → the forest says **approve**. Tree 3 was 'wrong' for this person, but the group outvoted it – that is the error-cancelling effect.

PART 2 · HOW IT WORKS

# 7. Hands-on: build the idea from scratch

The video builds a *toy* forest of three trees by hand to make the idea concrete. It is not exactly how scikit-learn does it internally, but it shows every concept. Below is a clean, corrected version of that notebook.

## 7.1 Dataset and the three sampling functions

```
import numpy as np, pandas as pd
from sklearn.datasets import make_classification
from sklearn.tree import DecisionTreeClassifier

X, y = make_classification(n_samples=100, n_features=5, n_informative=3, random_state=0)
df = pd.DataFrame(X, columns=[f"f{i}" for i in range(5)]); df["target"] = y   # 100 rows, 6 columns

def sample_rows(df, frac):                      # ROW sampling (with replacement)
    return df.sample(int(len(df) * frac), replace=True)

def sample_features(df, frac):                  # COLUMN sampling (target column is never sampled!)
    feats = df.columns[:-1]
    k = int(len(feats) * frac)                  # 0.5 * 5 = 2.5 -> 2 columns
    chosen = np.random.choice(feats, k, replace=False)
    return df[list(chosen) + ["target"]]        # always keep the target

def combined_sampling(df, row_frac, col_frac):  # ROWS + COLUMNS
    return sample_features(sample_rows(df, row_frac), col_frac)
```

**CLEARED UP FROM THE VIDEO**

Bugs the presenter hit live (and the fixes built into the code above):

• the column percentage must be computed on **(number of columns − 1)**, because the target column must not be counted;

• the function must **return the target column too**, otherwise you cannot train;

• the slice used for training must match the number of input columns (0:5 for five features, 0:4 for four) – use *df.columns[:-1]* instead of hard-coding numbers.

## 7.2 Experiment A – row sampling

```
dfs = [sample_rows(df, 0.3) for _ in range(3)]           # 30 % of 100 rows each -> shape (30, 6)
trees = [DecisionTreeClassifier().fit(d.iloc[:, :-1], d["target"]) for d in dfs]

q = df.iloc[[5], :-1]                                    # a "new" query point
votes = [t.predict(q)[0] for t in trees]                 # e.g. [0, 0, 1]
print(votes, "->", np.bincount(votes).argmax())          # majority vote
```

**What the video observed:** with 10 % of the rows (10 rows) the three trees are tiny, and re-running gives different trees each time (e.g. a 6-4 split one time, 5-5 the next) – randomness at work. With 30

% the trees get bigger and the differences become easier to see. For the query point two trees said **0** and one said **1**, so the majority prediction was **0** – the correct answer.

## 7.3 Experiment B – column sampling

```
dfs = [sample_features(df, 0.5) for _ in range(3)]         # 2 of the 5 input columns each
for d in dfs: print(d.columns.tolist(), d.shape)           # e.g. ['f1','f4','target'] (100, 3)

trees = [DecisionTreeClassifier().fit(d.iloc[:, :-1], d["target"]) for d in dfs]
# each tree must be queried with ITS OWN columns:
votes = [t.predict(q[d.columns[:-1]])[0] for t, d in zip(trees, dfs)]
```

With *frac = 0.5* the result has 100 rows × 2 input columns; with *0.8* it has 4 input columns (int(0.8 × 5) = 4). The video printed *df3.columns* to prove that all three datasets had different columns, which is why the trees looked different. Again two trees voted **1** and one voted **0**, so the forest answered **1** (correct).

## 7.4 Experiment C – combined sampling

```
dfs = [combined_sampling(df, 0.5, 0.5) for _ in range(3)]  # 50 rows x (2 inputs + target)
print(dfs[0].shape)                                        # (50, 3)
```

Shape **(50, 3)**: 50 rows, 2 input columns and 1 target column. Three trees were trained, each on different rows *and* columns; the query was answered 1, 0, 1 → majority **1**.

> **TIP**
> Run the notebook several times. Every run produces different trees and sometimes different votes. You will get a real feeling for how internal randomness creates diversity – and how voting stabilises the answer.

PART 3 · WHY IT WORKS

# 8. Why does it work? Bias and Variance

Random Forest's strong performance is hidden in the **bias–variance tradeoff**. Every prediction error of a model is made of three parts:

$$\textit{Expected error} = \textit{Bias}^2 + \textit{Variance} + \textit{Irreducible noise } (\sigma^2)$$

| Term | Plain meaning | Symptom |
|---|---|---|
| **Bias** | The model is too simple / did not learn well – it is wrong even on the **training data**. | Underfitting: high training error |
| **Variance** | The model memorised the training data and its noise. Change the data slightly and the output changes a lot. | Overfitting: low training error, high test error |
| **Noise** | Randomness in the data itself that no model can predict. | Cannot be removed |

We always want **low bias and low variance**. The problem: they usually move in **opposite directions**. Reduce one and the other tends to grow.

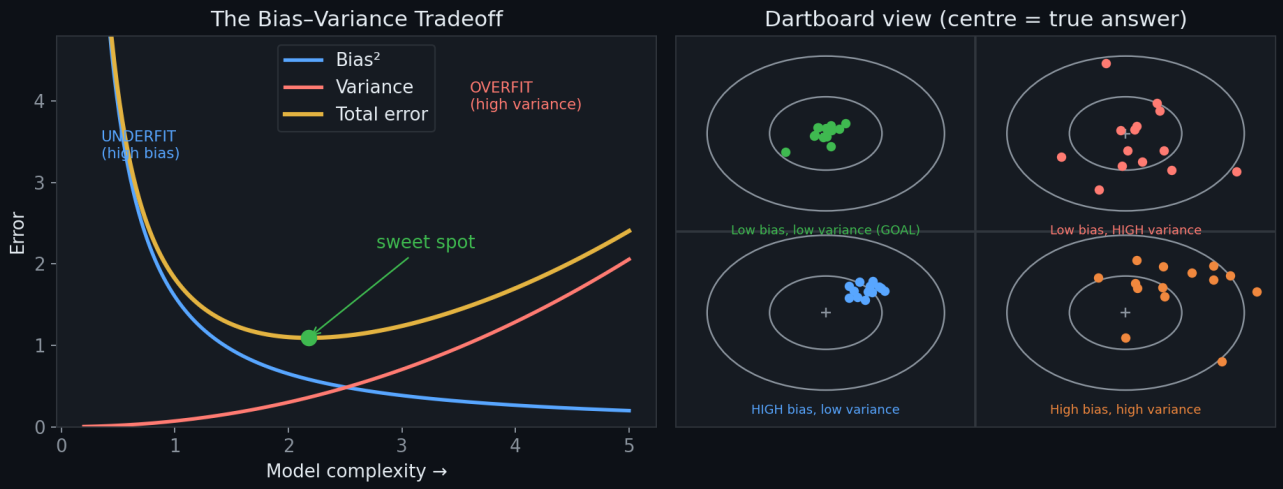



*Figure 5 – Left: as model complexity grows, bias² falls but variance rises; the total error is lowest at a sweet spot. Right: the dartboard view – the centre is the truth.*

## 8.1 Algorithms grouped by their bias/variance behaviour

| Class | Examples | Behaviour |
|---|---|---|
| Low bias · **High variance** | Fully grown Decision Tree (no max_depth), SVM (flexible kernels), KNN with a very small k | Great on training data, worse on new data |
| **High bias** · Low variance | Linear Regression, (often) Logistic Regression | Stable but too simple to capture complex patterns |
| Low bias · Low variance | **← the goal. Almost no single algorithm gives it …** | … |

**KEY IDEA**

**This is exactly where Random Forest helps.** Give it a low-bias / high-variance algorithm (a fully grown tree) and it returns a **low-bias / low-variance** model: it keeps the bias as it is and **reduces the variance**.

PART 3 · WHY IT WORKS

# 9. How Random Forest reduces variance

## 9.1 The intuition – spreading the noise

Each base tree is fully grown, so each has **low bias and high variance**: no mistakes on its own training data but mistakes on new data. Now give each tree only a random part of the data (say 1,000 of 10,000 rows) – a tree still has low bias.

What happens to variance? Imagine we replace 100 of the 10,000 points with very noisy outliers.

- **Single tree:** all 100 noisy points hit the one tree, which bends itself around them – the result changes a lot.
- **Forest:** the noisy points are drawn randomly, so one tree may get 10 of them, another 5, another 0, another 15. **No single tree takes the full hit; the damage is spread out and each tree's share is small.** After averaging, the prediction barely changes → variance is reduced.

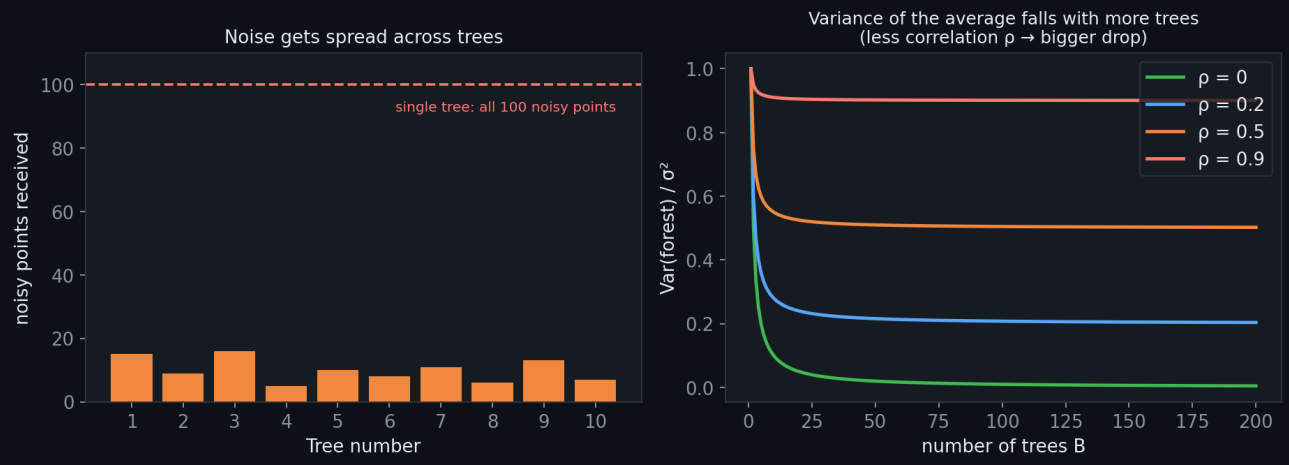



*Figure 6 – Left: 100 noisy points are spread over 10 trees instead of hurting one tree. Right: the variance of the average shrinks as trees are added.*

## 9.2 The mathematics (pro level)

Suppose each tree has prediction variance $\sigma^2$ and the correlation between any two trees is $\rho$. The variance of the average of B trees is:

$$Var\left(\frac{1}{B}\sum h_b(x)\right) = \rho\cdot\sigma^2 + \frac{(1-\rho)}{B}\cdot\sigma^2$$

Read it term by term:

- The second term **vanishes as B grows** – this is why **more trees help** (up to a point).
- The first term, **$\rho\sigma^2$**, does not disappear. It depends on how **correlated** the trees are. If all trees were identical ($\rho = 1$), the forest would be no better than one tree.
- So we must make the trees **as different as possible** – lower $\rho$. This is the reason for bootstrap sampling *and* for random feature selection at every split (Chapter 11).

**WHY THIS MATTERS**

Special cases: ρ = 0 → Var = σ²/B (perfect 1/B shrink). ρ = 1 → Var = σ² (no benefit).

Bias of the average = bias of a single tree (all trees are statistically alike), which is why averaging does **not** increase the bias.

Also the reason to keep trees **deep** inside a forest: deep trees have low bias; the averaging takes care of the variance.

PART 3 · WHY IT WORKS

# 10. Seeing it: decision boundaries and regression

## 10.1 Classification on concentric circles

The video generated 500 points with *make_circles* (two classes, concentric circles, with noise) and compared a fully grown Decision Tree with a Random Forest of 500 trees by plotting their **decision boundaries**.

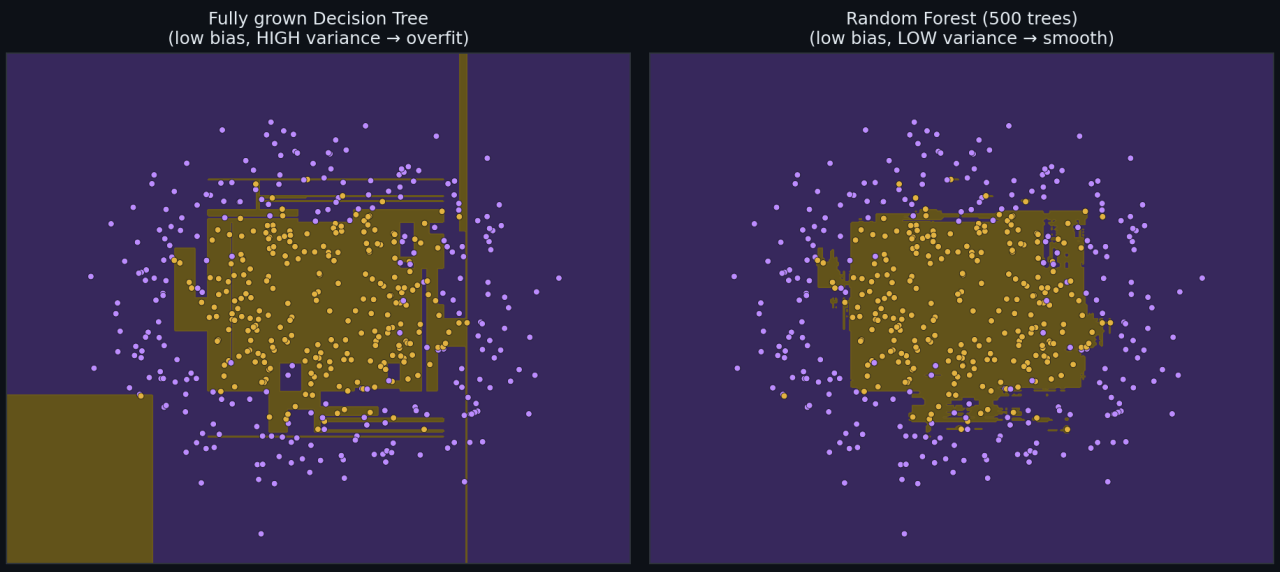



*Figure 7 – Left: the single tree creates tiny isolated regions around single points (overfitting). Right: the forest produces one smooth, realistic boundary.*

```
from sklearn.datasets import make_circles
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
X, y = make_circles(n_samples=500, noise=0.2, factor=0.5, random_state=1)
dt = DecisionTreeClassifier().fit(X, y)                              # fully grown tree
rf = RandomForestClassifier(n_estimators=500, random_state=0).fit(X, y)
# then predict on a dense grid (np.meshgrid) and use plt.contourf to draw each boundary
```

**Reading the left plot:** the long thin regions exist only because of a single stray point. A human sees that the area around should belong to the other class, but the tree bent itself to get that one training point right → **low bias, high variance**.

**Reading the right plot:** training error rises slightly (bias goes up a little – the forest happily 'mislabels' some noisy training points), but the unnecessary regions disappear and the model captures the true shape of the data. Test error and variance drop considerably → **low bias, low variance**.

## 10.2 Regression on noisy data

Data: one input column and one output; a red curve shows the **true pattern** and the dots are that curve plus random noise. A Decision Tree Regressor tries to touch every dot – including outliers – so its curve zig-zags (low training error, high variance). A forest of 1,000 trees produces a smooth curve that stays near the red one.

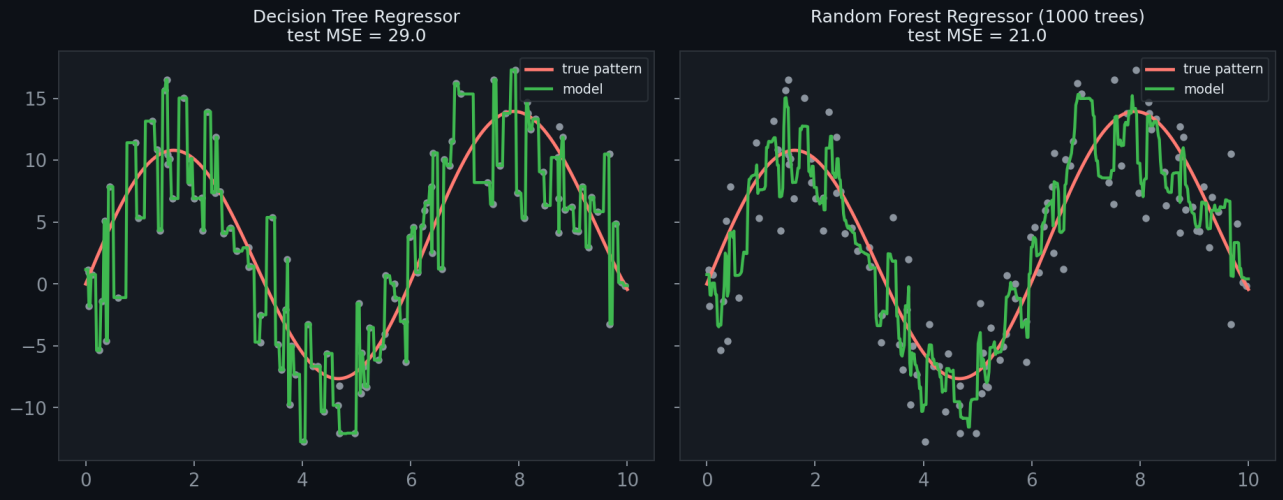



*Figure 8 – Test MSE of the single tree = 29.0 vs forest = 21.0 in this run. (The presenter's data gave about 22.7 vs 15 – the exact numbers change with data and seed; the ordering does not.)*

$$MSE = (1/n)\ \Sigma\ (\ y_i - \hat{y}_i\ )^2$$

**TIP**

A lower MSE means the predictions are closer to the truth. Using a forest instead of a tree, the error dropped by roughly a quarter to a third – with no tuning at all.

# 11. Bagging vs Random Forest

A favourite interview question. Random Forest is an implementation of bagging, but there are **two** differences, and most candidates only know the first.

## 11.1 Difference 1 – the base algorithm

- **Bagging:** general technique – the base estimators can be Decision Trees, KNN, SVM or anything.
- **Random Forest:** base estimators are **only Decision Trees**.

You can see this in scikit-learn. *BaggingClassifier* has a parameter to choose the base algorithm (*estimator*; called *base_estimator* in older versions; default is a Decision Tree). *RandomForestClassifier* has *n_estimators* but **no** parameter to choose another base algorithm.

## 11.2 Is 'Bagging with Decision Trees' then a Random Forest? NO.

This is the follow-up question that traps many people. There is a second difference, in **how columns are sampled**.

| | Bagging | Random Forest |
|---|---|---|
| Where are columns sampled? | Once, at the **tree level**, before the tree is built | Again at **every node** (every split) |
| Consequence | Only the chosen columns (e.g. X0 and X1) can ever be used anywhere in that tree | Different columns are available at every node, so over the whole tree all columns can appear |
| Randomness / diversity | Less | **More** → trees are less correlated (lower ρ) |
| Typical result | Good | Usually better |




*Figure 9 – Tree-level sampling (left) vs node-level sampling (right). In Bagging every node chooses between X0 and X1 only; in a Random Forest each node gets a fresh random pair.*

## 11.3 Example with 5 columns and 2 columns per tree

- **Bagging:** tree 1 gets columns {1, 3}: *every* split in that tree uses column 1 or 3. Tree 2 gets {4, 5}, tree 3 gets {1, 4}...
- **Random Forest:** at the first node 2 random columns {2, 3} are offered, so the split is made on 2 or 3. For the next node a fresh random draw from all 5 columns gives {4, 5}; the next gives {2, 1} ... The parameter that controls the number is *max_features*.

```
from sklearn.ensemble import BaggingClassifier, RandomForestClassifier
from sklearn.tree import DecisionTreeClassifier, plot_tree

# Bagging with trees: column sampling happens ONCE per tree
bag = BaggingClassifier(DecisionTreeClassifier(), n_estimators=100, max_features=2).fit(X, y)
plot_tree(bag.estimators_[0])      # every split uses only X0 or X1  -> the same two columns

# Random Forest: column sampling happens at EVERY node
rf = RandomForestClassifier(n_estimators=100, max_features=2).fit(X, y)
plot_tree(rf.estimators_[0])       # splits use X1, X3, X2, X4 ... all five columns show up
```

**What the video showed:** in the Bagging tree, every node split on only X0 or X1. In the Random Forest tree, X1, X3, X2, X4 – all five columns – appeared. More randomness → more differences between the base models → better ensemble (as the theory says: the more different the models, the better the ensemble, provided each is still above 50 % accuracy).

**INTERVIEW**

**Q: What is the difference between Bagging and Random Forest?**

A: (1) Bagging can use any base algorithm; Random Forest uses only Decision Trees. (2) Even with trees, Bagging samples columns once per tree, whereas Random Forest samples a new random subset of columns at every node split, creating more randomness and decorrelated trees.

**Follow-up: 'If I build bagging with decision trees, is it a Random Forest?'** No – because of the node-level feature sampling.

**TIP**

Random Forest is not guaranteed to win – 'most of the time' it does. Try both and compare with cross-validation.

PART 5 · FREE VALIDATION

# 12. OOB (Out-of-Bag) evaluation

OOB evaluation is a neat concept found in almost all bagging-style ensembles, and a frequent interview topic. Its name: **Out-of-Bag**.

## 12.1 The idea

Take a dataset with 60 rows and train 5 trees with bootstrap = True (sampling **with replacement**). Each tree draws its rows one by one: say row 21, then 32, then 21 again … Some rows appear several times. But because of replacement, some rows are **never selected** for a given tree.

A row that was left out of a tree's bag is an **out-of-bag (OOB) sample** *for that tree*. That tree has never seen it – to the tree it is completely new data. So we can use those rows as a **free validation set**: for each row, ask only the trees that did not train on it, compare to the true label, and compute the accuracy. No separate hold-out set needed.

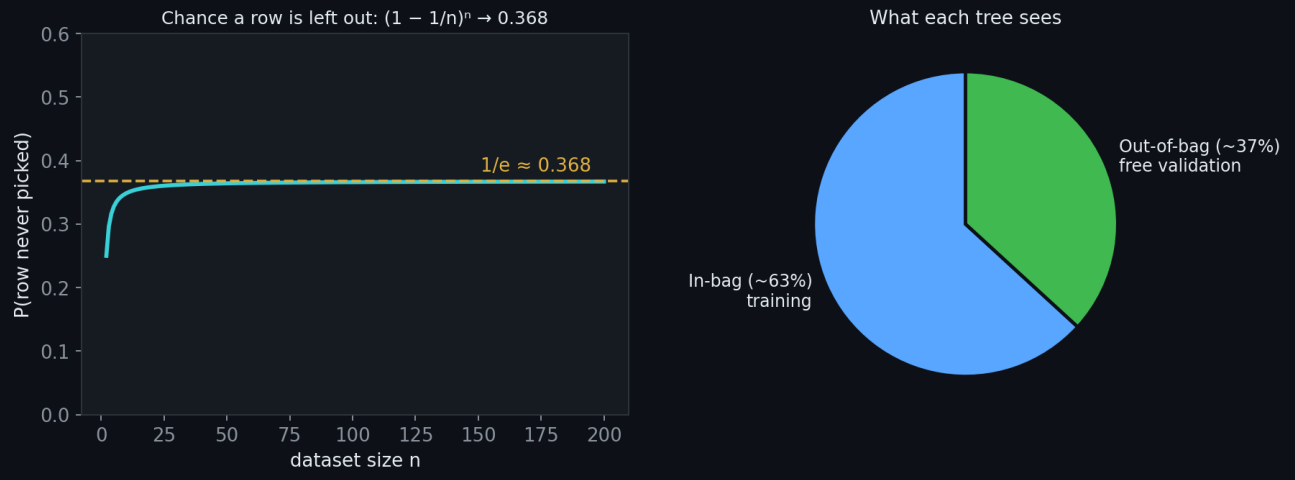



*Figure 10 – About 37 % of the rows are left out of every bootstrap sample (the curve converges to 1/e).*

## 12.2 Why ≈ 37 %? (the maths)

Draw N rows with replacement from N rows. In one draw a particular row is *not* chosen with probability (1 − 1/N). It must be missed N times in a row:

$$P(\text{row never chosen}) = (1 - 1/N)^N \to 1/e \approx 0.368 \text{ as } N \text{ grows}$$

*So ≈ 63.2 % of unique rows are in the bag and ≈* ***36.8 %*** *are out of the bag.*

**CLEARED UP FROM THE VIDEO**

The video quotes '37.2' and '37.7' while describing it; the exact limit is **36.8 % ≈ 37 %**. For small datasets the figure is slightly different (e.g. N = 10 gives 34.9 %).

## 12.3 Using it in scikit-learn

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=1)
```

```
rf = RandomForestClassifier(n_estimators=200, bootstrap=True, oob_score=True, random_state=1)  # oob_score=True is REQU
rf.fit(X_train, y_train)

print("OOB accuracy :", rf.oob_score_)                      # estimate from out-of-bag rows
print("Test accuracy:", accuracy_score(y_test, rf.predict(X_test)))
```

In the video's heart-disease example (a very small dataset of only a few dozen people) the OOB score was **0.80** while accuracy on the separate test set was **0.60**. The numbers differ, but OOB gives a rough, honest estimate of how the model will perform. The difference is large here only because the dataset is tiny – with a bigger dataset the two numbers agree closely.

**WATCH OUT**

• *oob_score=True* only works with *bootstrap=True* and needs enough trees so that every row is out-of-bag at least once (use ≥ 50–100 trees).

• OOB is an estimate on the training data; it replaces a validation set, but keep a final test set for the last check.

**INTERVIEW**

**Q: What is OOB score?** A: In bootstrap sampling ≈ 37 % of the rows are not used to train a given tree. These unseen rows are used to evaluate the ensemble without needing a separate validation set.

PART 6 · FEATURE IMPORTANCE

# 13. Feature importance

Decision Trees, Random Forests, Bagging – and later AdaBoost, Gradient Boosting and XGBoost – can all tell you **how important each column (feature) was** for the predictions. This chapter covers: (1) what it is and why you need it, (2) how to get it in code, (3) the maths behind it (Chapter 14).

## 13.1 What is it – and why do we need it?

Feature importance gives every column a score. A higher score means the algorithm relied on that column more. Uses:

- **Feature selection** – too many columns slow training and can hurt the model. Keep the best features, drop the rest.
- **Interpretability** – clients ask *why*. For a loan model: 'Why was my application rejected?' Feature importance shows which factors (e.g. credit score, income) drove the decisions.
- **Insight** – learn what really matters in your data.

## 13.2 Visual example – MNIST digits

In the MNIST dataset every image is 28 × 28 = 784 pixels, and every pixel is one **column** (so a row = one image, with a label telling the digit). Think about where digits are written: mostly in the **centre**. The corners and borders are almost always blank, so they cannot help to separate a 3 from an 8. A Random Forest discovers this automatically.




*Figure 11 – Pixel importance from a Random Forest. (Shown with scikit-learn's built-in 8 × 8 digits so the notebook runs offline; the presenter's 28 × 28 MNIST heat-map shows the same thing – bright centre, dark border.)*

```
import seaborn as sns, matplotlib.pyplot as plt
from sklearn.ensemble import RandomForestClassifier

rf = RandomForestClassifier().fit(X_train, y_train)    # X has 784 pixel columns (MNIST)
imp = rf.feature_importances_                          # one number per column
print(imp.shape)                                       # (784,)
sns.heatmap(imp.reshape(28, 28))                       # reshape back to the image shape
plt.show()                                             # centre pixels light up, borders stay dark
```

**INFO**

*feature_importances_* is normalised: **all scores add up to 1**. The video's heat-map shows that the central pixels matter far more than the border pixels; the darker the shade, the less important the pixel.

PART 6 · FEATURE IMPORTANCE

# 14. The maths of feature importance – step by step

To understand a forest's importances, first understand a **single tree**. scikit-learn defines it as 'the total reduction of the criterion brought by that feature' – also called **Gini importance** (Mean Decrease in Impurity).

## 14.1 The formulas

$$\text{Node importance: } ni(t) = (n_t/N)\cdot G_t - (n_R/N)\cdot G_R - (n_L/N)\cdot G_L$$
$$\text{Feature importance: } FI(j) = \Sigma_{t \text{ splits on } j}\, ni(t) \,/\, \Sigma_{\text{all nodes } t}\, ni(t)$$

| Symbol | Meaning |
|---|---|
| $n_t$ | number of samples reaching node t |
| N | total number of training samples ($n_T$ in the video) |
| $G_t$ | impurity (Gini or entropy) at node t |
| $n_L$, $n_R$, $G_L$, $G_R$ | samples and impurity of the left / right child |

Recipe: **(1)** compute the importance of every split node; **(2)** for each feature, add the importances of all nodes that split on it; **(3)** divide by the total over all nodes so everything sums to 1.

## 14.2 Worked example 1 – five rows, two columns

The video trains a tree on 5 rows. Node A splits on column 1; node B (4 samples) splits on column 0. Both children of node B are pure.

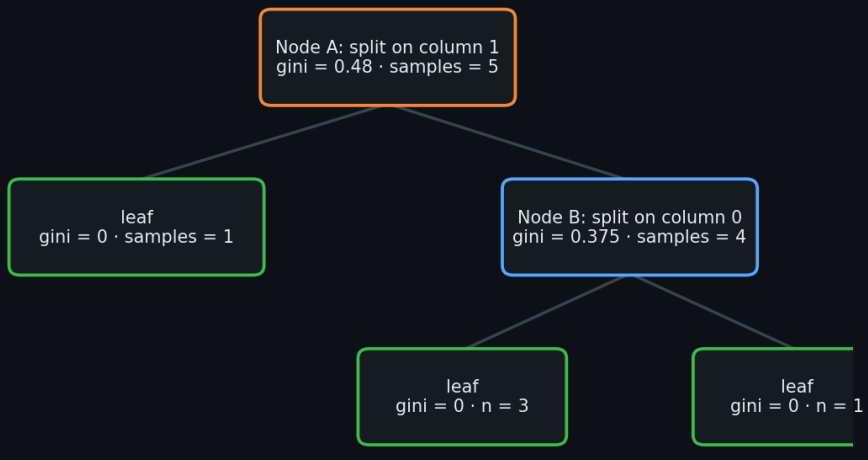



*Figure 12 – The first tree. Node A: samples = 5, Gini = 0.48. Node B: samples = 4, Gini = 0.375.*

**CALCULATION**

**Node A (column 1):**

ni(A) = (5/5)·0.48 − (4/5)·0.375 − (1/5)·0 = 0.48 − 0.30 = **0.18**

**Node B (column 0):** both children are pure (Gini 0)

ni(B) = (4/5)·0.375 − 0 − 0 = **0.30**

**Total** = 0.18 + 0.30 = 0.48

**FI(column 0) = 0.30 / 0.48 = 0.625**

**FI(column 1) = 0.18 / 0.48 = 0.375** (sum = 1 ✓)

Checking with scikit-learn's *tree.feature_importances_* gives the same numbers: column 0 is more important (≈ 0.6) than column 1. Column 0 wins because it cleaned the bigger, messier group (4 samples) completely.

## 14.3 Worked example 2 – three split nodes

With 15 samples the tree has three split nodes. Node A splits on column 1; nodes B and C both split on column 0.

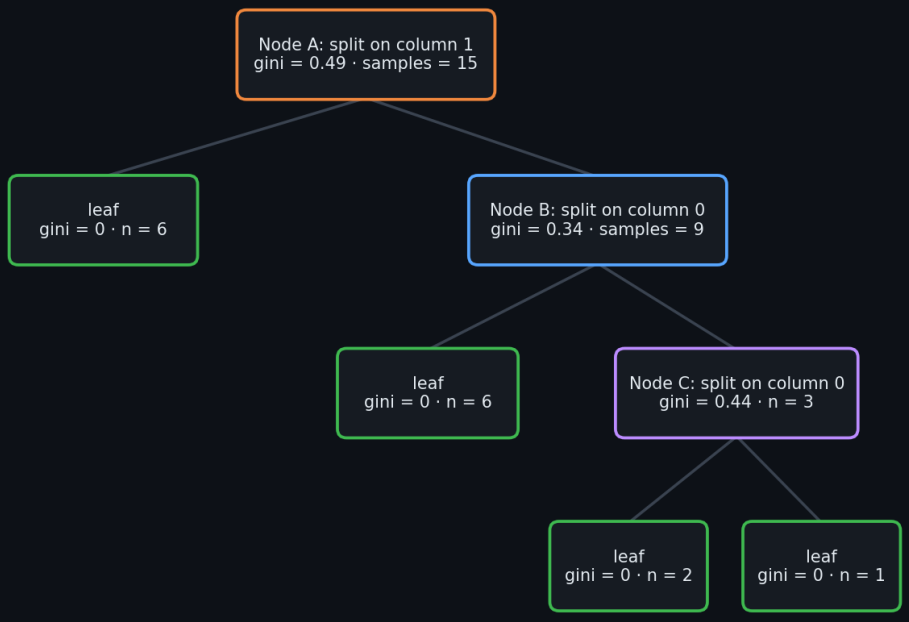



*Figure 13 – Second tree (15 samples). Values read from the plotted tree (rounded).*

**CALCULATION**

ni(A) = (15/15)·0.49 − (9/15)·0.34 − (6/15)·0 = 0.49 − 0.204 = **0.286**

ni(B) = (9/15)·0.34 − (3/15)·0.44 − (6/15)·0 = 0.204 − 0.088 = **0.116**

ni(C) = (3/15)·0.44 − 0 − 0 = **0.088**

Total = 0.286 + 0.116 + 0.088 = **0.490** (note: it equals the root impurity when all leaves are pure – a nice sanity check)

**FI(column 0) = (0.116 + 0.088) / 0.490 = 0.416**

**FI(column 1) = 0.286 / 0.490 = 0.584**

**CLEARED UP FROM THE VIDEO**

In the video, node B was rounded to 0.1 (instead of 0.116), giving 0.39 / 0.61. The unrounded values above give 0.42 / 0.58 – the conclusion (column 1 slightly more important) is identical. Always carry more decimals!

## 14.4 From one tree to a forest – just average

Every tree in the forest computes its own feature importances. The forest simply **averages** them:

$$FI_{forest}(j) = (1 / B) \cdot \Sigma_{b=1..B}\, FI_{tree\ b}(j)$$

**EXAMPLE**

Two trees. Tree 1 gives column 0 → 1.00 and column 1 → 0.00. Tree 2 gives column 0 → 0.55 and column 1 → 0.44.

Forest: column 0 = (1.00 + 0.55) / 2 = **0.775**; column 1 = (0.00 + 0.44) / 2 = **0.22**.




*Figure 14 – A Random Forest does nothing special: it averages the importances computed by the individual trees.*

```
rf = RandomForestClassifier(n_estimators=2).fit(X, y)
print(rf.estimators_[0].feature_importances_)     # importances of tree 1
print(rf.estimators_[1].feature_importances_)     # importances of tree 2
print(rf.feature_importances_)                    # = their average
```

## 14.5 A warning – impurity importance can mislead

**WATCH OUT**

Impurity-based importances are **biased toward high-cardinality features** (columns with many unique values, like IDs or continuous numbers) – they offer more places to split, so they look important even when they are not. They are also computed on the **training** data.

Better alternative: **permutation importance** – shuffle one column in the test set and measure how much the score drops. A big drop = important feature.

```
from sklearn.inspection import permutation_importance
r = permutation_importance(rf, X_test, y_test, n_repeats=10, random_state=0)
for name, m, s in zip(X.columns, r.importances_mean, r.importances_std):
    print(f"{name:15s} {m:.3f} +/- {s:.3f}")
```

PART 7 · PRO LEVEL

# 15. Pro level – hyper-parameters

Random Forest works well with defaults, but knowing the knobs lets you squeeze out more accuracy and control overfitting and speed. Names are from scikit-learn's *RandomForestClassifier / Regressor*.

| Parameter | Default | What it does / how to use it |
|---|---|---|
| **n_estimators** | 100 | Number of trees. More = more stable (variance ↓) but slower. Gains flatten out; 200–500 is typical. It does **not** cause overfitting. |
| **max_features** | 'sqrt' (clf) · 1.0 (reg) | Columns considered at **each split**. The key randomness knob. Lower → trees more different (ρ ↓) but each individually weaker. Try sqrt, log2, 0.3–0.6. |
| **max_depth** | None | Depth limit. None = fully grown (low bias – what RF wants). Limit it to speed up or reduce memory. |
| **min_samples_leaf** | 1 | Minimum samples per leaf. Raise (3–10) to smooth predictions and reduce noise-fitting. |
| **min_samples_split** | 2 | Minimum samples needed to split a node. |
| **max_samples** | None (= N) | Size of each bootstrap sample (needs bootstrap=True). Smaller = faster, more diverse. |
| **bootstrap** | True | Row sampling with replacement. False = every tree sees all rows (only column randomness). |
| **oob_score** | False | Compute the OOB estimate (Chapter 12). |
| **criterion** | 'gini' | Impurity measure: gini or entropy (classification); squared_error (regression). |
| **class_weight** | None | 'balanced' helps with imbalanced classes. |
| **max_leaf_nodes · ccp_alpha** | None · 0 | Alternative ways to limit tree size / prune. |
| **n_jobs** | None | -1 uses all CPU cores. Trees are independent, so training parallelises perfectly. |
| **random_state** | None | Fix for reproducible results. |




*Figure 15 – Breast-cancer dataset: accuracy rises then plateaus as trees are added; OOB accuracy tracks the test accuracy closely.*

PART 7 · PRO LEVEL

# 16. Tuning and a complete pipeline

Use cross-validation, never a single lucky split. *RandomizedSearchCV* is usually enough; *GridSearchCV* is exhaustive.

```
from sklearn.datasets import load_breast_cancer
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split, RandomizedSearchCV
from sklearn.metrics import classification_report

X, y = load_breast_cancer(return_X_y=True, as_frame=True)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.2, stratify=y, random_state=42)

# 1) Baseline with defaults (often already very good)
rf = RandomForestClassifier(n_estimators=300, oob_score=True, n_jobs=-1, random_state=42).fit(X_tr, y_tr)
print("OOB:", round(rf.oob_score_, 3), " Test:", round(rf.score(X_te, y_te), 3))

# 2) Tune
param_dist = {
    "n_estimators":     [200, 300, 500, 800],
    "max_features":     ["sqrt", "log2", 0.3, 0.5],
    "max_depth":        [None, 8, 12, 20],
    "min_samples_leaf": [1, 2, 4, 8],
    "min_samples_split":[2, 5, 10],
    "bootstrap":        [True],
}
search = RandomizedSearchCV(RandomForestClassifier(n_jobs=-1, random_state=42), param_dist,
                            n_iter=25, cv=5, scoring="accuracy", random_state=42, n_jobs=-1)
search.fit(X_tr, y_tr)
print("Best params:", search.best_params_, " CV score:", round(search.best_score_, 3))

# 3) Final evaluation on untouched test data + importances
best = search.best_estimator_
print(classification_report(y_te, best.predict(X_te)))
print(sorted(zip(best.feature_importances_, X.columns), reverse=True)[:5])
```

**TIP**

**Tuning order that works:** (1) start with defaults; (2) raise n_estimators until the score stops improving; (3) tune max_features; (4) tune min_samples_leaf / max_depth to control overfitting; (5) check OOB vs test for sanity.

PART 7 · PRO LEVEL

# 17. Strengths, limits and related models

| Strengths | Limitations |
|---|---|
| Strong accuracy with little tuning | Larger and slower than one tree; many trees use memory |
| Handles classification and regression | Less interpretable than a single tree (use feature importance / SHAP) |
| Few assumptions; no feature scaling needed | Cannot extrapolate beyond the training range in regression |
| Handles non-linear patterns and interactions | Impurity importance can be biased (Chapter 14.5) |
| Robust to noise and outliers; low variance | Often slightly behind tuned gradient boosting (XGBoost / LightGBM) on tabular data |
| Built-in OOB estimate and feature importance | Very sparse / very high-dimensional text data: linear models may be better |
| Trains in parallel (n_jobs=-1) | |

## 17.1 Related models

- **Extra-Trees** (*ExtraTreesClassifier*) – even more randomness: split thresholds are also chosen at random. Faster, sometimes more accurate.
- **Gradient Boosting / XGBoost / LightGBM** – boosting builds trees *sequentially* and mainly reduces **bias**; Random Forest builds them in parallel and reduces **variance**.
- **Isolation Forest** – anomaly detection with random trees.

| | Random Forest (bagging) | Gradient Boosting (boosting) |
|---|---|---|
| Trees built | In parallel, independently | One after another, each fixing the previous errors |
| Main effect | Reduces variance | Reduces bias (and variance with regularisation) |
| Tree depth | Deep | Shallow |
| Tuning need | Low | Higher |
| Overfitting risk | Low | Higher if over-trained |

PART 7 · PRO LEVEL

# 18. Cheat sheet, interview Q&A; and glossary

## 18.1 One-page summary

| Topic | Remember |
|---|---|
| Definition | Many decision trees trained on random subsets; vote (clf) / mean (reg) |
| Name | Forest = many trees · Random = random sampling of rows and columns |
| Step 1 – Bootstrap | Rows with replacement (+ random columns at each node) |
| Step 2 – Aggregate | Majority vote / average |
| Why it works | Fully grown trees have low bias, high variance; averaging cuts variance: Var = $\rho\sigma^2$ + $(1-\rho)\sigma^2/B$ |
| Bagging vs RF | Any base model vs trees only · column sampling per tree vs per node |
| OOB | ≈ 36.8 % rows unused per tree → free validation (oob_score=True) |
| Feature importance | Σ node impurity decrease per feature, normalised, averaged over trees; use permutation importance for reliability |
| Key knobs | n_estimators, max_features, max_depth, min_samples_leaf |

## 18.2 Interview questions

| Question | Short answer |
|---|---|
| **What is a Random Forest?** | An ensemble of decision trees trained by bagging (bootstrap rows + random features at each split) whose outputs are aggregated by voting or averaging. |
| **Why is it called 'random'?** | Randomness in the selection of rows (bootstrap) and of the features considered at every split. |
| **How does it reduce overfitting?** | Single deep trees have high variance. Averaging many decorrelated trees keeps the bias but cuts the variance. |
| **Does adding more trees cause overfitting?** | No. Variance falls and then plateaus; it only costs time and memory. |
| **Difference between Bagging and Random Forest?** | Base learner can be anything vs trees only; and RF samples features at each node rather than once per tree. |
| **What is OOB?** | About 37 % of rows are not in a given tree's bootstrap sample; evaluating on them gives a validation estimate without a hold-out set. |
| **How is feature importance computed?** | Total impurity reduction a feature brings across the tree's nodes (weighted by samples), normalised; the forest averages over trees. |
| **Weakness of impurity importance?** | Bias to high-cardinality features and computed on training data – prefer permutation importance. |
| **Do you need to scale features?** | No, trees are insensitive to monotonic scaling. |
| **Random Forest vs Gradient Boosting?** | Parallel + variance reduction vs sequential + bias reduction. |

## 18.3 Glossary

| Term | Meaning |
|---|---|
| Base estimator | One of the small models inside the ensemble (here: a decision tree) |
| Bootstrap | Sampling rows with replacement |
| Aggregation | Combining predictions (vote / mean) |
| Bias | Error from a model that is too simple; high training error |
| Variance | Error from sensitivity to the training data; overfitting |
| Gini impurity | $1 - \Sigma p^2$; 0 = pure node |
| OOB sample | Row not used to train a particular tree |
| Feature importance | Score of how useful each column was |
| Cardinality | Number of unique values in a column |
| ρ (rho) | Correlation between trees – lower is better |

**WHERE TO GO NEXT**

You now know Random Forest from the basic idea (many different trees voting), through the mechanics (bootstrapping + aggregation), the theory (bias–variance, $\text{Var} = \rho\sigma^2 + (1-\rho)\sigma^2/B$), the extras (OOB, feature importance with full maths) to professional tuning. The best next step: run every code block in this tutorial yourself and change one number at a time.